

LanceJob

Plateforme de Mise en Relation Freelancers-Clients

Rapport Technique Complet

Majidi

11 juin 2025

Table des matières

1 Introduction

1.1 Présentation du Projet

LanceJob est une plateforme web moderne conçue pour faciliter la mise en relation entre freelancers et clients. Cette application permet aux professionnels indépendants de proposer leurs services et aux clients de trouver les compétences adaptées à leurs besoins.

1.2 Objectifs

- Créer une interface intuitive pour la gestion des projets freelance
- Implémenter un système de matching intelligent entre offres et demandes
- Assurer la sécurité des transactions et des données utilisateurs
- Optimiser l'expérience utilisateur sur tous les appareils

2 Architecture du Système

2.1 Architecture Générale

Le projet LanceJob adopte une architecture moderne basée sur une séparation claire entre le frontend et le backend :

- **Frontend** : Application React.js avec TypeScript
- **Backend** : API REST développée avec Node.js et Express
- **Base de données** : MongoDB pour la persistance des données
- **Authentification** : JWT (JSON Web Tokens)

2.2 Structure des Dossiers

```
1 lancejob/  
2   frontend/  
3     public/  
4     src/  
5       components/  
6       pages/  
7       services/  
8       types/  
9       utils/  
10    package.json  
11    tsconfig.json  
12  backend/  
13    src/  
14      controllers/  
15      models/  
16      routes/  
17      middleware/  
18      utils/  
19    package.json  
20    server.js  
21  reports/  
22    main.tex
```

Listing 1 – Structure du projet

3 Conception et Modélisation

3.1 Modèle de Données

3.1.1 Entité Utilisateur

- **Champs principaux** : ID, nom, email, mot de passe, type (freelancer/client)
- **Profil freelancer** : compétences, portfolio, tarifs, disponibilité
- **Profil client** : entreprise, secteur d'activité, projets publiés

3.1.2 Entité Projet

- **Informations générales** : titre, description, budget, délais
- **Compétences requises** : technologies, niveau d'expertise
- **Statut** : ouvert, en cours, terminé, annulé

3.1.3 Entité Candidature

- **Relation** : freelancer projet
- **Données** : proposition tarifaire, délai proposé, message
- **Statut** : en attente, acceptée, refusée

3.2 Diagrammes UML

3.2.1 Diagramme de Classes

Le système est organisé autour de trois classes principales :

- **User** : gestion des utilisateurs et authentification
- **Project** : gestion des projets et missions
- **Application** : gestion des candidatures

3.2.2 Diagramme de Séquence

Les interactions principales incluent :

1. Inscription/Connexion utilisateur
2. Publication d'un projet par un client
3. Candidature d'un freelancer
4. Sélection et attribution du projet

4 Technologies Utilisées

4.1 Frontend

- **React.js 18** : Framework JavaScript pour l'interface utilisateur
- **TypeScript** : Typage statique pour une meilleure robustesse
- **Material-UI** : Bibliothèque de composants pour un design moderne
- **React Router** : Gestion de la navigation
- **Axios** : Client HTTP pour les appels API

4.2 Backend

- **Node.js** : Environnement d'exécution JavaScript côté serveur
- **Express.js** : Framework web minimaliste et flexible
- **MongoDB** : Base de données NoSQL orientée documents
- **Mongoose** : ODM (Object Document Mapping) pour MongoDB
- **bcrypt** : Hachage sécurisé des mots de passe
- **jsonwebtoken** : Implémentation des tokens JWT

4.3 Outils de Développement

- **Git** : Système de contrôle de version
- **npm** : Gestionnaire de packages
- **ESLint** : Analyse statique du code
- **Prettier** : Formatage automatique du code

5 Implémentation

5.1 Configuration du Backend

5.1.1 Serveur Express

```
1 const express = require('express');
2 const cors = require('cors');
3 const mongoose = require('mongoose');
4
5 const app = express();
6
7 // Middleware
8 app.use(cors());
9 app.use(express.json());
10
11 // Routes
12 app.use('/api/auth', authRoutes);
13 app.use('/api/projects', projectRoutes);
14 app.use('/api/users', userRoutes);
15
16 // Connexion MongoDB
17 mongoose.connect(process.env.MONGODB_URI)
18   .then(() => console.log('Connect MongoDB'))
19   .catch(err => console.error('Erreur de connexion:', err));
```

Listing 2 – Configuration du serveur

5.1.2 Modèles Mongoose

```
1 const userSchema = new mongoose.Schema({
2   name: { type: String, required: true },
3   email: { type: String, required: true, unique: true },
4   password: { type: String, required: true },
5   role: { type: String, enum: ['freelancer', 'client'], required: true },
6   profile: {
```

```
7   skills: [String],
8   hourlyRate: Number,
9   availability: String,
10  portfolio: [String]
11  }
12 }, { timestamps: true });
```

Listing 3 – Modèle Utilisateur

5.2 Configuration du Frontend

5.2.1 Composants React

```
1 import React, { useState } from 'react';
2 import { TextField, Button, Paper } from '@mui/material';
3 import { useAuth } from '../hooks/useAuth';
4
5 const LoginForm: React.FC = () => {
6   const [credentials, setCredentials] = useState({
7     email: '',
8     password: ''
9   });
10  const { login } = useAuth();
11
12  const handleSubmit = async (e: React.FormEvent) => {
13    e.preventDefault();
14    await login(credentials);
15  };
16
17  return (
18    <Paper elevation={3} sx={{ p: 4 }}>
19      <form onSubmit={handleSubmit}>
20        <TextField
21          fullWidth
22          label="Email"
23          value={credentials.email}
24          onChange={(e) => setCredentials({
25            ...credentials,
26            email: e.target.value
27          })}
28        />
29        <Button type="submit" variant="contained">
30          Se connecter
31        </Button>
32      </form>
33    </Paper>
34  );
35 };
```

Listing 4 – Composant de connexion

6 Sécurité

6.1 Authentification

— Hachage des mots de passe : Utilisation de bcrypt avec salt

- **Tokens JWT** : Authentification stateless sécurisée
- **Middleware d'authentification** : Protection des routes sensibles

6.2 Validation des Données

- Validation côté client avec des schémas TypeScript
- Validation côté serveur avec Mongoose
- Sanitisation des entrées utilisateur

6.3 Protection CORS

Configuration CORS restrictive pour limiter les accès depuis des domaines non autorisés.

7 Tests

7.1 Tests Unitaires

- **Frontend** : Jest et React Testing Library
- **Backend** : Jest et Supertest pour les tests d'API

7.2 Tests d'Intégration

- Tests des flux complets utilisateur
- Tests de l'API avec une base de données de test

7.3 Tests End-to-End

- Cypress pour les tests d'interface utilisateur
- Scénarios de test complets : inscription, connexion, création de projet

8 Déploiement

8.1 Environnements

- **Développement** : Serveurs locaux avec hot reload
- **Test** : Environnement de staging avec données de test
- **Production** : Déploiement sur services cloud

8.2 CI/CD

Pipeline automatisé incluant :

1. Exécution des tests
2. Build des applications
3. Déploiement automatique
4. Vérifications post-déploiement

9 Performance et Optimisation

9.1 Frontend

- Code splitting avec React.lazy
- Optimisation des images
- Mise en cache des requêtes API
- Progressive Web App (PWA)

9.2 Backend

- Indexation des champs MongoDB fréquemment interrogés
- Pagination des résultats
- Mise en cache Redis pour les données fréquentes
- Compression gzip des réponses

10 Monitoring et Maintenance

10.1 Logging

- Winston pour la journalisation côté serveur
- Niveaux de log appropriés (error, warn, info, debug)
- Rotation des fichiers de log

10.2 Monitoring

- Surveillance des performances applicatives
- Monitoring de la base de données
- Alertes en cas de dysfonctionnement

11 Évolutions Futures

11.1 Fonctionnalités Prévues

- Système de notation et d'avis
- Chat en temps réel entre freelancers et clients
- Système de paiement intégré
- Application mobile native

11.2 Améliorations Techniques

- Migration vers Next.js pour le SSR
- Implémentation de GraphQL
- Microservices pour une meilleure scalabilité
- Intelligence artificielle pour le matching

12 Conclusion

Le projet LanceJob représente une solution complète et moderne pour la mise en relation entre freelancers et clients. L'architecture choisie assure une séparation claire des responsabilités, une sécurité robuste et une expérience utilisateur optimale.

Les technologies utilisées garantissent la scalabilité et la maintenabilité du système, tandis que les pratiques de développement adoptées (tests, CI/CD, monitoring) assurent la qualité et la fiabilité de la plateforme.

12.1 Retour d'Expérience

Ce projet a permis de mettre en pratique les meilleures pratiques du développement web moderne, de la conception à la mise en production, en passant par les tests et le déploiement.